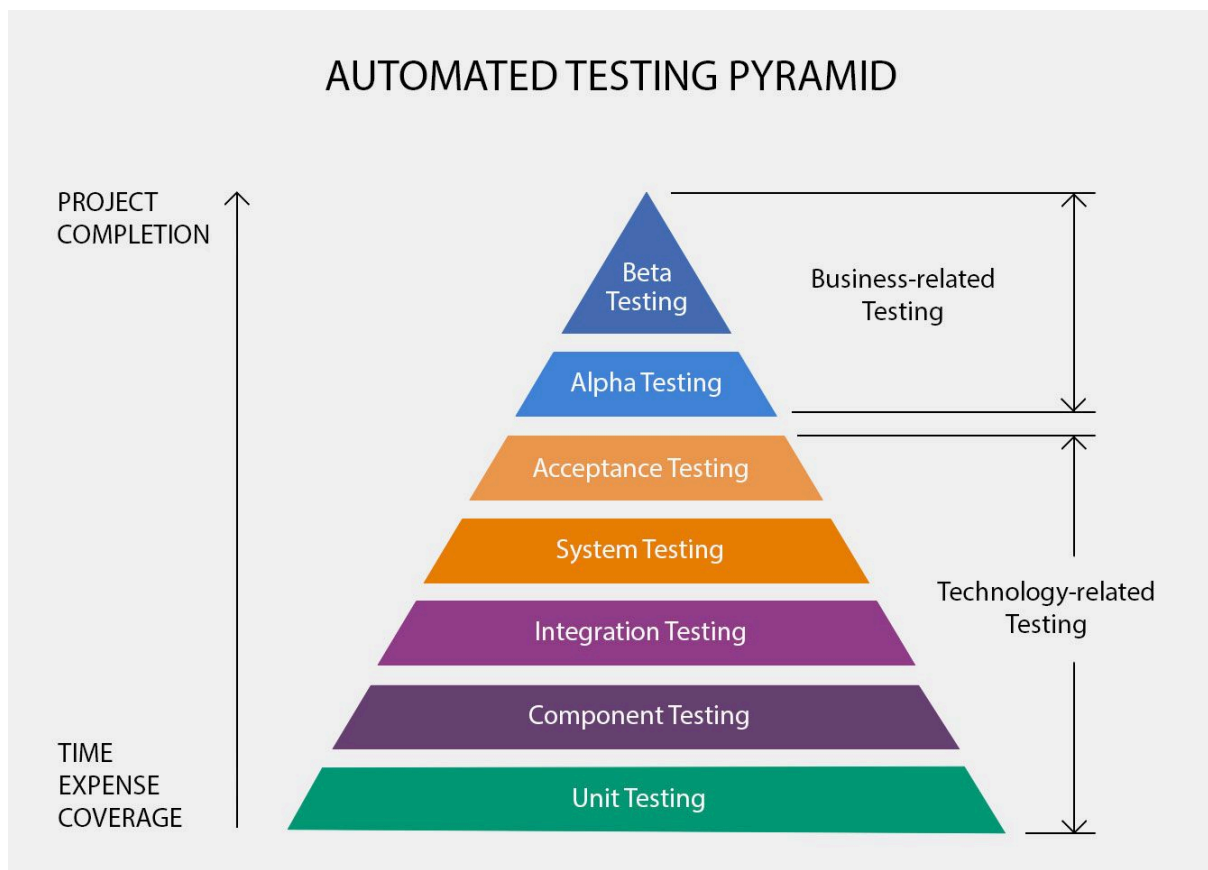
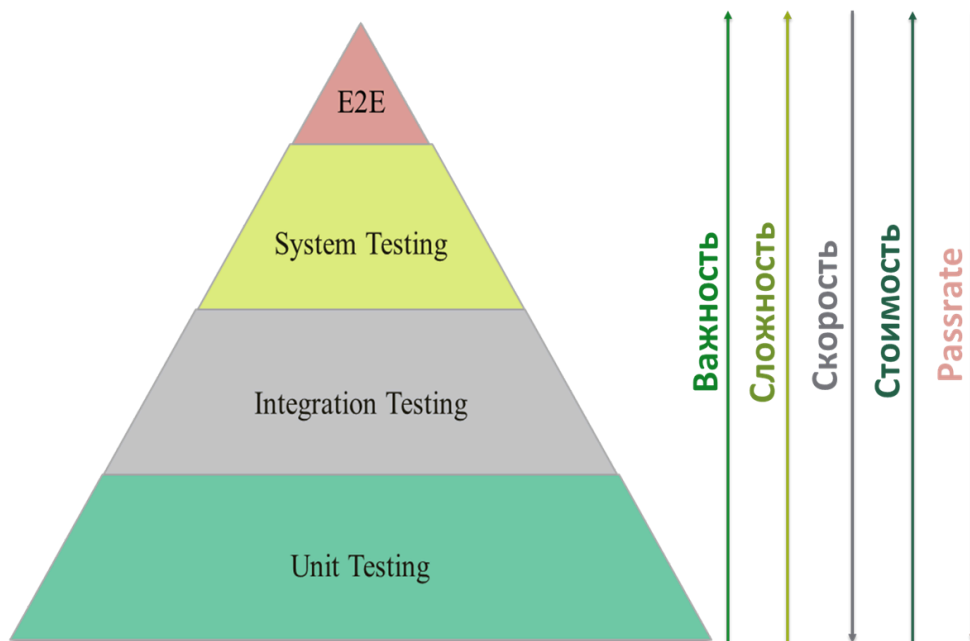
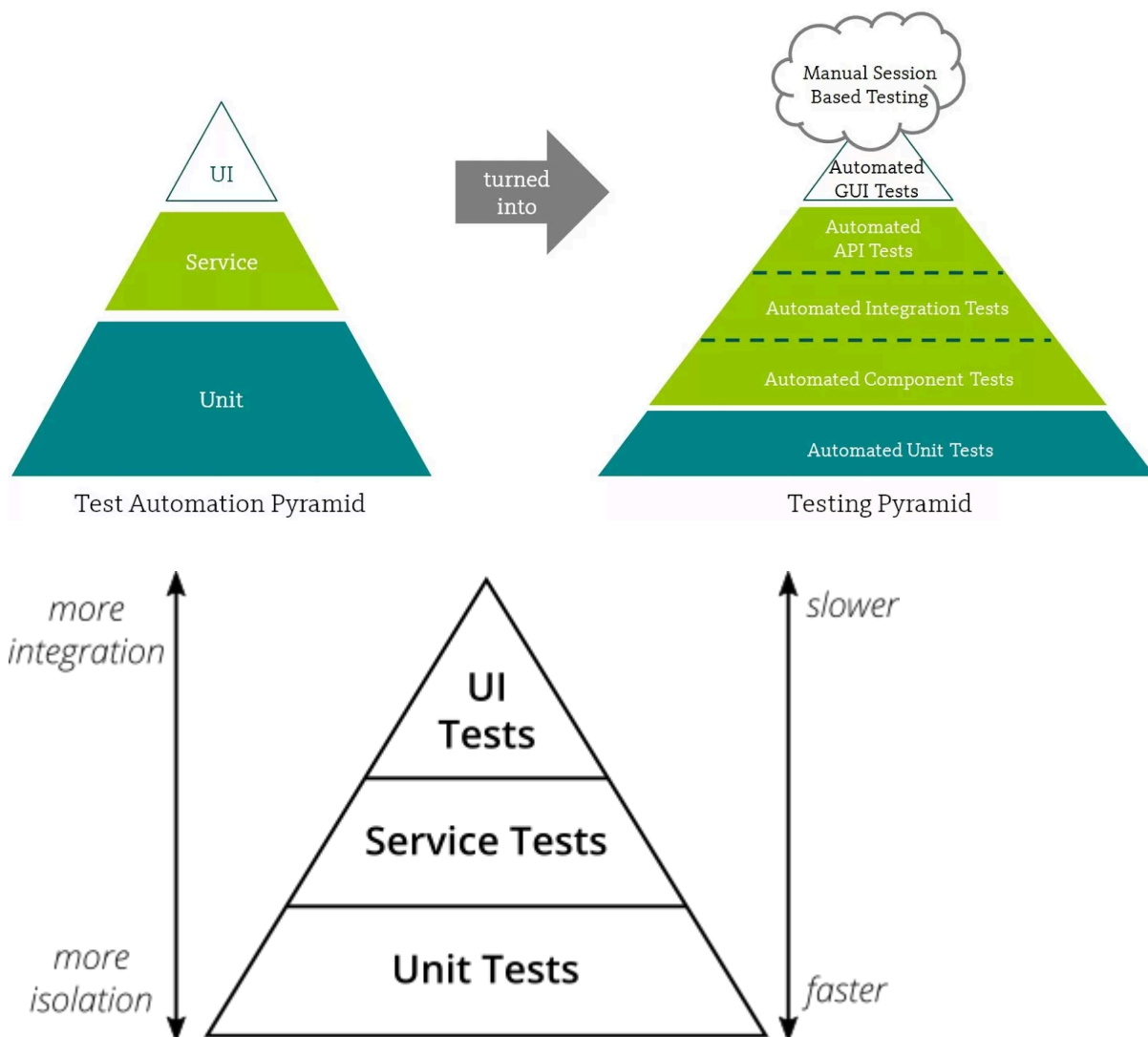


Лекция 7

Автоматизация тестирования



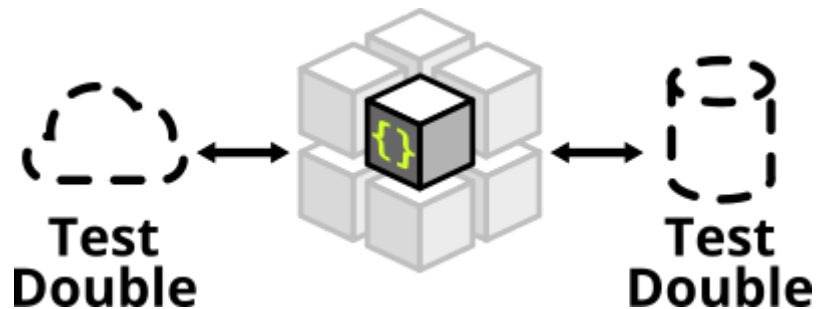


«Пирамида тестов» — метафора, которая означает группировку тестов программного обеспечения по разным уровням детализации. Она также даёт представление, сколько тестов должно быть в каждой из этих групп. Несмотря на то, что концепция тестовой пирамиды существует довольно давно, многие команды разработчиков по-прежнему пытаются неправильно реализовать её на практике должным образом.

Более подробно можно почитать здесь: <https://habr.com/ru/articles/358950/> и <https://martinfowler.com/bliki/UnitTest.html>

Юнит-тесты

Основа набора тестов состоит из юнит-тестов (модульных тестов). Они проверяют, что отдельный юнит (*тестируемый субъект*) кодовой базы работает должным образом. Модульные тесты имеют максимально узкую область среди всех тестов в наборе тестов. Количество юнит-тестов в наборе значительно превышает количество любых других тестов.



1. Модульные тесты являются низкоуровневыми, фокусирующими на небольшой части программной системы.
2. Модульные тесты пишутся самими программистами с использованием их обычных инструментов - единственное отличие заключается в использовании какой-либо структуры модульного тестирования.
3. Модульные тесты должны быть значительно быстрее других видов тестов.

Если вы пишете на функциональном языке, то юнитом скорее всего будет отдельная функция. Ваши юнит-тесты вызовут функцию с различными параметрами и обеспечат возврат ожидаемых значений. В объектно-ориентированном языке юнит может варьироваться от отдельного метода до целого класса.

Общительные и одинокие тесты

Некоторые утверждают, что всех участников (например, вызываемые классы) тестируемого субъекта следует заменить на **имитации (mocks)** или **заглушки (stubs)**, чтобы создать идеальную изоляцию, избежать побочных эффектов и сложной настройки теста. Другие утверждают, что на имитации и заглушки следует заменять только участников, которые замедляют тест или проявляют сильные побочные эффекты (например, классы с доступом к БД или сетевыми вызовами).

Иногда эти два вида юнит-тестов называют **одинокими (solitary)** в случае тотального применения имитаций и заглушек или **общительными (sociable)** в случае реальных коммуникаций с другими участниками.

Имитации и заглушки

Имитации (mocks) и заглушки (stubs) — это два разных типа тестовых дублёров. Вы заменяете реальную вещь (например, класс, модуль или функцию) поддельной копией. Подделка выглядит и действует как оригинал (даёт такие же ответы на те же вызовы методов), но это заранее установленные ответы, которые вы сами определяете для юнит-теста.

Разработка через тестирование (Test-driven development (TDD)) — это способ написания кода , который включает в себя написание автоматизированного тестового случая на уровне модуля , который не проходит, затем написание кода, достаточного для прохождения теста, затем рефакторинг как тестового кода, так и производственного кода, а затем повторение с другим новым тестовым случаем.

Структура теста

Хорошая структура всех тестов (не только модульных) такова:

1. Настройка тестовых данных.
2. Вызов тестируемого метода.
3. Проверка, что возвращаются ожидаемые результаты.

Есть хорошая мнемоника для запоминания этой структуры: **три А (Arrange, Act, Assert)**.

```
[TestMethod]
public void AddWithInc_2Plus3Inc1_Returned6()
{
    // arrange
    var calc = new Calculator();
    double arg1 = 2;
    double arg2 = 3;
    double expected = 6;
    // act
    double result = calc.AddWithInc(arg1, arg2);
    // assert
    Assert.AreEqual(expected, result);
}
```

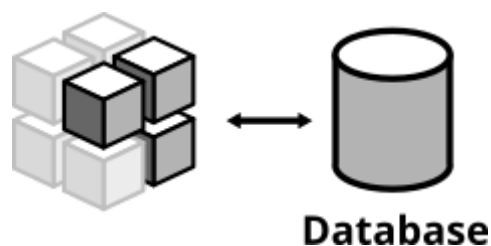
Интеграционные тесты

Все нетривиальные приложения интегрированы с некоторыми другими частями (базы данных, файловые системы, сетевые вызовы к другим приложениям). В юнит-тестах вы обычно имитируете их для лучшей изоляции и повышения скорости. Тем не менее, ваше приложение будет реально взаимодействовать с другими частями — и это следует протестировать. Для этого предназначены **интеграционные тесты**. Они проверяют интеграцию приложения со всеми компонентами вне приложения.

Для автоматизированных тестов это означает, что нужно запустить не только собственное приложение, но и интегрируемый компонент. Если вы тестируете интеграцию с БД, то при выполнении тестов надо запустить БД. Чтобы проверить чтение файлов с диска нужно сохранить файл на диск и загрузить его в интеграционный тест.

Интеграционные тесты живут на границе сервиса. Концептуально они всегда запускают действие, которое приводит к интеграции с внешней частью (файловой системой, базой данных, отдельным сервисом).

Тест интеграции БД выглядит следующим образом:



1. Запуск базы данных.
2. Подключение приложения к БД.
3. Запуск функции в коде, которая записывает данные в БД.
4. Проверка, что ожидаемые данные записаны в базу путём их чтения из БД.

Другой пример. Тест интеграции сервиса с отдельной службой через REST API может выглядеть следующим образом:



1. Запуск приложения.
2. Запуск инстанса отдельной службы (или тестового дублёра с тем же интерфейсом).
3. Запуск функции в коде, которая считывает данные из API внешней службы.
4. Проверка, что приложение правильно разбирает ответ.

При написании *узких интеграционных тестов* стремитесь локально запускать внешние зависимости: локальную базу данных MySQL, тест на локальной файловой системе ext4. Если интегрируетесь с отдельной службой, то или запустите экземпляр этой службы локально, или создайте и запустите поддельную версию, которая имитирует поведение реальной службы.

Если нет возможности локально запустить стороннюю службу, то лучше запустить выделенный тестовый инстанс и указать на него в интеграционном тесте. В автоматизированных тестах избегайте интеграции с реальной системой продакшна. Запуск тысяч тестовых запросов на систему продакшна — верный способ разозлить людей, потому что вы забиваете их логи (в лучшем случае) или просто ддосите их сервис (в худшем случае). Интеграция с сервисом по сети — типичное свойство *широкого интеграционного теста*. Обычно из-за неё тесты труднее писать и они медленнее работают.

Контрактные тесты

Более современные компании нашли способ масштабирования разработки путём распределения работ среди разных команд. Они создают отдельные, слабо связанные службы, не мешая друг на другу, и интегрируют их в большую, цельную систему. Именно с этим связана недавняя шумиха вокруг микросервисов.

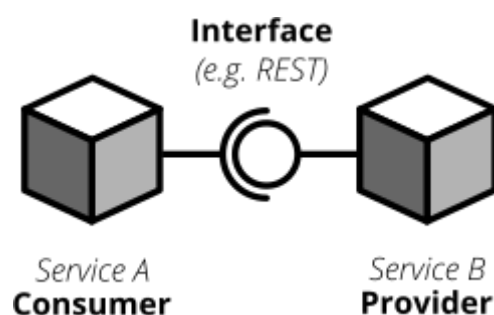
Разделение системы на множество небольших сервисов часто означает, что эти сервисы должны взаимодействовать друг с другом через определённые (желательно чётко определенные, но иногда случайно созданные) интерфейсы.

Интерфейсы между разными приложениями могут быть реализованы в разных форматах и технологиях. Самые распространённые:

- REST и JSON через HTTPS;
- RPC с использованием чего-то вроде [gRPC](#);
- построение событийно-ориентированной архитектуры с использованием очередей.

Каждый интерфейс задействует две стороны: поставщика и потребителя.

Поставщик предоставляет данные потребителям. Потребитель обрабатывает данные, полученные от поставщика. В мире REST поставщик создаёт REST API со всеми необходимыми конечными точками, а потребитель обращается к этому REST API, чтобы получить данные или инициировать изменения в другой службе. В асинхронном мире событийно-ориентированной архитектуры поставщик (часто именуемый издателем) публикует данные в очередь; а потребитель (часто называемый подписчиком) подписывается на эти очереди, считывает и обрабатывает данные.

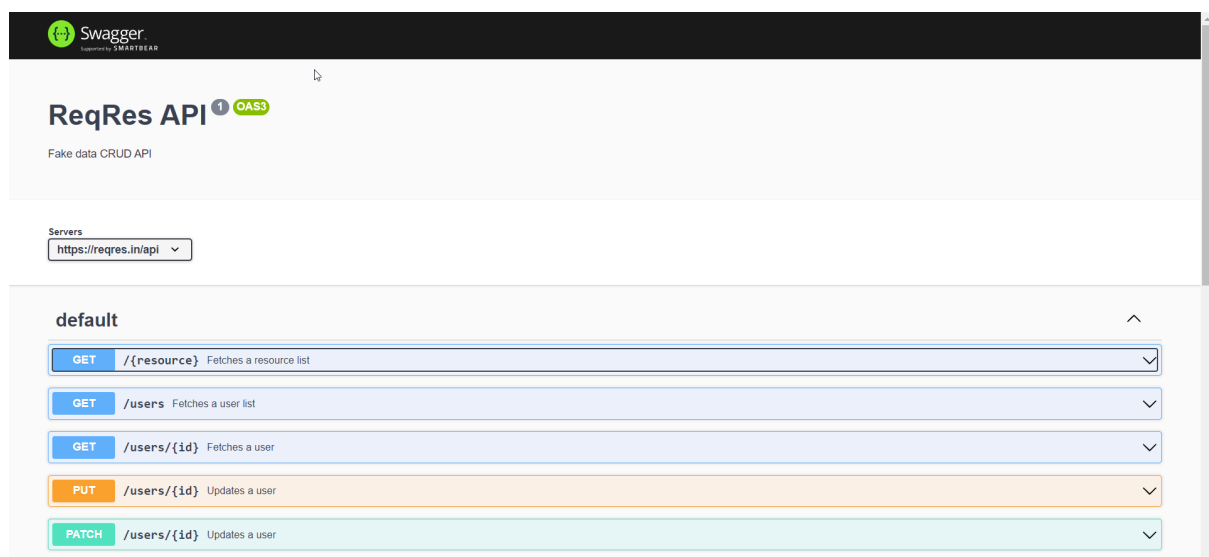


Поскольку сервисы поставщика и потребителя распределяются по разным командам, то вы оказываетесь в ситуации, когда нужно чётко указать

интерфейс между ними (так называемый контракт). Традиционно компании подходят к этой проблеме следующим образом:

1. Написать длинную и подробную спецификацию интерфейса (*контракт*).
2. Реализовать сервис поставщика согласно определённому контракту.
3. Передать спецификации интерфейса стороне потребителя.
4. Подождать, пока они реализуют свою часть интерфейса.
5. Запустить крупномасштабный ручной системный тест, чтобы всё проверить.
6. Надеяться, что обе команды будут всегда соблюдать определения интерфейса и не облажаются.

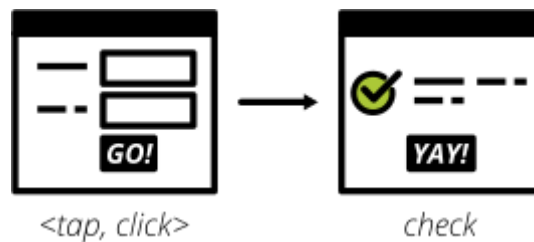
Ссылка на пример контракта : <https://regres.in/api-docs/>



Тесты UI

1. *Web тестирование*
2. *Тестирование мобильных приложений*
3. *Тестирование десктопных приложений*

Тесты UI проверяют правильность работы пользовательского интерфейса приложения. Действия пользователя должны инициировать правильные события, данные должны представляться пользователю, состояние UI должно изменяться ожидаемым образом.



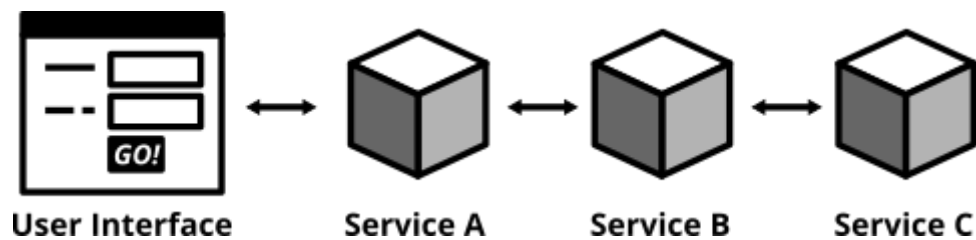
Тестирование пользовательского интерфейса не обязательно должно проводиться в сквозном режиме. В зависимости от используемой технологии, тестирование UI может оказаться таким же простым, как написание некоторых модульных тестов для фронтенда JavaScript с заглушенным бэкендом.

Для тестирования UI традиционных веб-приложений предназначены специальные инструменты вроде [Selenium](#).

В веб-интерфейсах желательно проверить несколько аспектов UI, в том числе поведение, вёрстка, юзабилити, соблюдение фирменного стиля и др.

Сквозные тесты

Тестирование развёрнутого приложения через UI — это самый полный тест, какой только можно провести. Описанные выше тесты UI через WebDriver — хорошие примеры сквозных тестов.

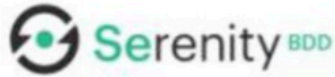


Сквозные тесты (также называемые [тестами широкого стека](#)) дают максимальную уверенность, работает программное обеспечение или нет. [Selenium](#) и [протокол WebDriver](#) позволяют автоматизировать тесты, автоматически отправляя headless-браузер на развёрнутые сервисы для выполнения кликов, ввода данных и проверки состояния UI. Можно использовать Selenium напрямую или применить инструменты на его основе, такие как [Nightwatch](#).

Сквозные тесты известны своей ненадёжностью, сбоями по неожиданным и непредвиденным причинам. Довольно часто это ложноположительные сбои. Чем более сложный UI, тем более хрупкими становятся тесты. Причуды браузера, проблемы с синхронизацией, анимация и неожиданные всплывающие диалоги — лишь некоторые из причин, из-за которых тратится больше времени на отладку, чем хотелось бы.

Инструменты для работы с web

Web



Инструменты для работы с мобильными приложениями

Mobile



Инструменты для работы с десктопными приложениями

Инструменты



Поиск элементов



Фреймворк

Поиск элементов

Критерии	UISPY	Inspect	FlauiInspect, Testcomplete UI и т.д
.NET Framework	.NET 2	.NET 2 .NET 3	.NET 2 .NET 3
Недостатки	Нет поддержки	Находит не все свойства элементов	Находит не все свойства элементов
	устарел	Постоянно зависает	Работает только со своим fw

Фреймворки

«Нестандартные» фреймворки

1. SikuliX
2. Robot framework

UI library

1. FlaUI

Selenium WebDriver

1. WinApp+Appium
2. Winium

Платные фреймворки



Дополнительная информация к автоматизации тестирования:

<https://infostart.ru/1c/articles/1086369/>

Лучшие практики и стратегии

1. **Тестируйте небольшие блоки кода:** Пишите тесты для небольших изолированных блоков работы (например, отдельных методов).
2. **Используйте осмысленные имена:** Убедитесь, что ваши методы тестирования имеют осмысленные имена, которые четко объясняют, для чего они предназначены.
3. **Шаблон Arrange-Act-Assert:** Шаблон Arrange-Act-Assert организует ваши тесты в три этапа:
 - Организовать: Настройте тест.
 - Действие: Вызовите тестируемый метод.
 - Assert: Проверьте результат.
4. **Имитируйте внешние зависимости:** Используйте mocks для имитации взаимодействия с внешними службами, базами данных и API.
5. **Тестируйте граничные случаи:** Для обеспечения надежности всегда проверяйте граничные условия и граничные случаи.

Перейти по ссылкам и выполнить проекты с модульным тестированием.

Практическое задание

1. <https://learn.microsoft.com/ru-ru/training/modules/visual-studio-test-tools/?source=recommendations>

Домашнее задание

2. <https://vscode.ru/prog-lessons/modulnoe-testirovanie-v-visual-studio.html>
3. <https://newtechaudit.ru/unit-testi-na-c/>
4. <https://www.c-sharpcorner.com/article/a-basic-introduction-of-unit-test-for-beginners/>
5. <https://www.c-sharpcorner.com/article/the-ultimate-guide-to-writing-nunit-unit-tests-in-net-9/>

Что выбрать?

<https://testsigma.com/blog/nunit-vs-xunit/>

Справочник по тестированию на C# :

<https://learn.microsoft.com/ru-ru/visualstudio/test/run-unit-tests-with-test-explorer?view=vs-2022>